

ATIVIDADE 3

Lab. Engenharia Software - 2026

Nicolas Ferreira

Arthur Carvalho de Oliveira

Maycon Vinicius de Oliveira

1. Protótipo funcional mínimo

Como protótipo inicial decidimos desenvolver a tela de login do sistema Terra da Esperança. A tela permite realizar o acesso ao sistema através do e-mail e senha do usuário. Também é possível ser redirecionado a tela de cadastrar-se através do link apresentado em tela. Para desenvolvimento foram utilizadas as seguintes tecnologias:

Tecnologias utilizadas:

- HTML
- CSS
- JAVASCRIPT

Tela de Login:

Terra da Esperança

Por favor, realize seu Login.

Email

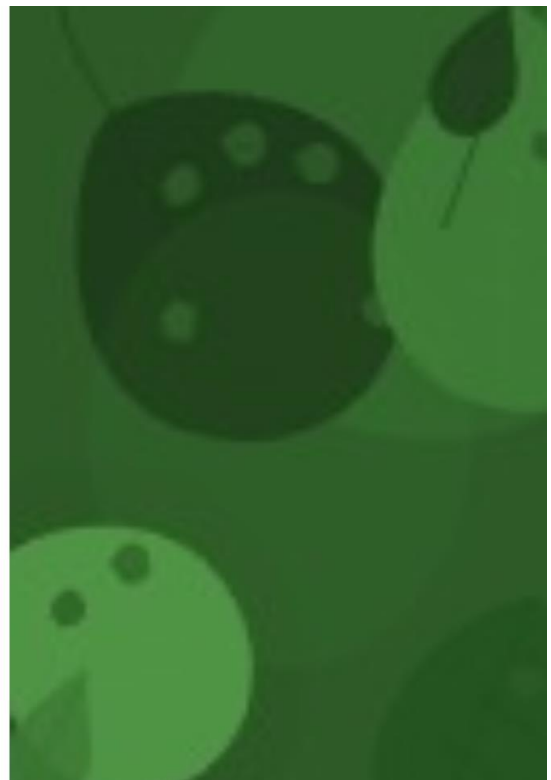
Coloque seu Email

Senha

Coloque sua Senha

Signup

Não possui conta? [Cadastrar-se](#)



Tela de Cadastro:

Terra da Esperança

Criar nova conta

Nome Completo

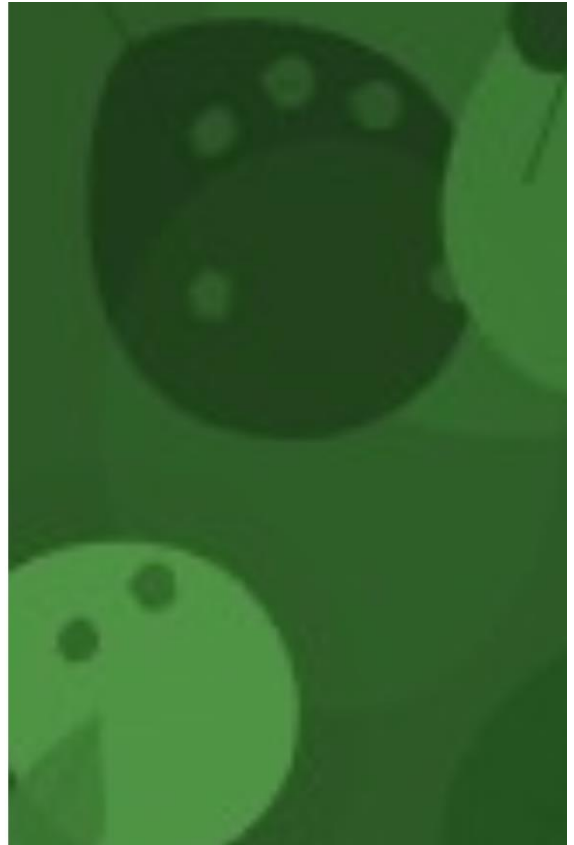
Email

Senha

Confirmar Senha

Cadastrar

Já possui conta? [Entrar](#)



Código:

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Login</title>
  <link
href="https://fonts.googleapis.com/css2?family=Roboto:wght@300;400;500;700&display=swap"
rel="stylesheet" />
  <style>
    *, ::before, ::after { box-sizing: border-box; margin: 0; padding: 0; }

    body {
      font-family: 'Roboto', sans-serif;
      height: 100vh;
      overflow: hidden;
      display: flex;
    }

    .left {
      flex: 0 0 58.33%;
      display: flex;
      align-items: center;
      justify-content: center;
      padding: 32px;
    }

    .form-wrap {
      width: 100%;
      max-width: 400px;
    }

    .title-app {
```

```

    font-size: 2.5rem;
    font-weight: 300;
    color: #1a1a1a;
    margin-bottom: 40px;
    line-height: 1.2;
}

.title-form {
    font-size: 1.5rem;
    font-weight: 700;
    color: #1a1a1a;
    margin-bottom: 24px;
}

.field {
    position: relative;
    margin-bottom: 16px;
}

.field label {
    position: absolute;
    top: -9px;
    left: 12px;
    background: #fff;
    padding: 0 4px;
    font-size: 0.75rem;
    color: rgba(0,0,0,0.6);
    pointer-events: none;
}

.field input {
    width: 100%;
    padding: 14px 14px;
    border: 1.5px solid rgba(0,0,0,0.23);
    border-radius: 4px;
    font-family: inherit;
    font-size: 1rem;
    color: #1a1a1a;
    background: #fff;
    outline: none;
    transition: border-color .2s;
}

.field input::placeholder { color: rgba(0,0,0,0.38); }
.field input:focus { border-color: #2d5a27; }
.field input:focus + label,
.field:focus-within label { color: #2d5a27; }

.btn-login {
    width: 100%;
    padding: 14px;
    background-color: #2d5a27;
    color: #fff;
    border: none;
    border-radius: 8px;
    font-family: inherit;
    font-size: 0.9375rem;
    font-weight: 600;
    cursor: pointer;
    margin-bottom: 16px;
    transition: background .2s;
    letter-spacing: 0.02em;
}

.btn-login:hover { background-color: #3d7a35; }

.footer-text {
    text-align: center;
    font-size: 0.875rem;
    color: rgba(0,0,0,0.6);
}

```

```

.footer-text a {
  color: #2d5a27;
  font-weight: 600;
  text-decoration: none;
}

.footer-text a:hover { text-decoration: underline; }

.right {
  flex: 1;
  min-height: 100vh;
  background: linear-gradient(145deg, #2d5a27 0%, #3d7a35 50%, #4e9444 100%);
  background-image: url('green-panel.svg');
  background-size: cover;
  background-position: center;
}

</style>
</head>
<body>

<div class="left">
  <div class="form-wrap">

    <h1 class="title-app">Terra da Esperança</h1>

    <h2 class="title-form">Por favor, realize seu Login.</h2>

    <div class="field">
      <input type="email" id="email" placeholder="Coloque seu Email" />
      <label for="email">Email</label>
    </div>

    <div class="field">
      <input type="password" id="password" placeholder="Coloque sua Senha" />
      <label for="password">Senha</label>
    </div>

    <button class="btn-login" onclick="handleSignup()">Signup</button>

    <p class="footer-text">
      Não possui conta?
      <a href="/login">Cadastrar-se</a>
    </p>

  </div>
</div>

<div class="right"></div>

<script>
  function handleSignup() {
    const email = document.getElementById('email').value;
    const password = document.getElementById('password').value;
    console.log('Login:', email, password);
  }
</script>

</body>
</html>

```

2. Estrutura inicial do projeto

Terra-da-Esperanca/

src/

model/

- Perfil.java
- Usuario.java
- Acolhido.java
- Triagem.java
- AcompanhamentoDiario.java
- InstituicaoParceira.java
- EmpresaParceira.java
- OportunidadeEmprego.java
- RelatorioMensal.java
- Doador.java
- Doacao.java
- Financa.java
- ItemEstoque.java
- AlertaEstoque.java
- ControleVagas.java

dao/

- PerfilDAO.java
- UsuarioDAO.java
- AcolhidoDAO.java
- TriagemDAO.java
- AcompanhamentoDiarioDAO.java
- InstituicaoParceiraDAO.java
- EmpresaParceiraDAO.java
- OportunidadeEmpregoDAO.java
- RelatorioMensalDAO.java
- DoadorDAO.java
- DoacaoDAO.java
- FinancaDAO.java
- ItemEstoqueDAO.java
- AlertaEstoqueDAO.java
- ControleVagasDAO.java

controller/

- AuthController.java
- AcolhidoController.java
- TriagemController.java
- AcompanhamentoController.java
- InstituicaoController.java
- EmpresaController.java
- OportunidadeController.java
- RelatorioController.java
- DoacaoController.java
- FinancaController.java
- EstoqueController.java
- VagasController.java

view/

- LoginView.java
- DashboardView.java
- AcolhidoView.java
- TriagemView.java
- AcompanhamentoView.java
- InstituicaoView.java
- EmpresaView.java
- OportunidadeView.java
- RelatorioView.java
- DoacaoView.java
- FinancaView.java
- EstoqueView.java
- VagasView.java

util/

- ConexaoDB.java
- Autenticacao.java
- ValidacaoCPF.java
- ValidacaoCNPJ.java

Main.java

```
├── database/SQL
│   ├── create_tables.sql
│   ├── inserts_iniciais.sql
│   └── views_relatorios.sql
└── README.md
```

3. Classes, componentes ou banco de dados

Para criação do projeto foi criado os seguintes scripts SQL:

```
CREATE TABLE tb_perfil (
  id_perfil          INTEGER,
  nome_perfil        VARCHAR(60)  UNIQUE NOT NULL,
  pode_visualizar    BOOLEAN      DEFAULT TRUE,
  pode_editar        BOOLEAN      DEFAULT FALSE,
  pode_excluir       BOOLEAN      DEFAULT FALSE,
  pode gerar_relatorio  BOOLEAN      DEFAULT FALSE,

  CONSTRAINT pk_tb_perfil_id_perfil PRIMARY KEY(id_perfil)
);

CREATE TABLE tb_usuarios (
  id_usuario        INTEGER,
  id_perfil          INTEGER,
  nome              VARCHAR(120) NOT NULL,
  cpf              CHAR(11)    UNIQUE NOT NULL,
  email            VARCHAR(150) UNIQUE NOT NULL,
  senha_hash       VARCHAR(255) NOT NULL,
  cargo            VARCHAR(80)  NOT NULL,
  especialidade     VARCHAR(80),
  jornada_diaria    TIME,
  data_inicio       DATE        NOT NULL,
  ativo            BOOLEAN      DEFAULT TRUE,

  CONSTRAINT pk_tb_usuarios_id_usuario PRIMARY KEY(id_usuario),
  CONSTRAINT fk_tb_usuarios_id_perfil  FOREIGN KEY(id_perfil)
  REFERENCES tb_perfil(id_perfil)
);

CREATE TABLE tb_acolhido (
  id_acolhido        INTEGER,
  id_usuario_resp     INTEGER,
  nome_completo       VARCHAR(100) NOT NULL,
  cpf                CHAR(11)    UNIQUE NOT NULL,
  data_nascimento     DATE        NOT NULL,
  data_admissao       DATE        NOT NULL,
  data_saida          DATE,
  status              VARCHAR(20) NOT NULL
                      CHECK(status IN ('ativo', 'inativo', 'fila_espera')),
  historico_reabilitacao TEXT,
  contato_emergencia  VARCHAR(120),
  telefone            VARCHAR(20),
  endereco            VARCHAR(255),
  escolaridade        VARCHAR(60),

  CONSTRAINT pk_tb_acolhido_id_acolhido
  PRIMARY KEY(id_acolhido),

  CONSTRAINT fk_tb_acolhido_id_usuario_resp
  FOREIGN KEY(id_usuario_resp)
  REFERENCES tb_usuarios(id_usuario)
);
```

```

CREATE TABLE tb_instituicao_parceira (
id_inst_parceira    INTEGER,
nome                VARCHAR(150) NOT NULL,
cnpj                CHAR(14)      UNIQUE NOT NULL,
telefone            VARCHAR(20),
email               VARCHAR(150),
endereco            VARCHAR(255),
vagas_disponiveis   INTEGER,
ativo               BOOLEAN        DEFAULT TRUE,

CONSTRAINT pk_tb_instituicao_parceira
PRIMARY KEY(id_inst_parceira)
);

CREATE TABLE tb_triagem (
id_triagem          INTEGER,
id_acolhido          INTEGER,
data_triagem         DATE          NOT NULL,
resultado            VARCHAR(20)   NOT NULL
                        CHECK(resultado IN ('aprovado', 'reprovado', 'fila_espera')),
pontuacao            DECIMAL(5,2),
observacoes          VARCHAR(500),
posicao_fila_espera   INTEGER,
id_inst_parceira      INTEGER,
id_usuario_resp       INTEGER      NOT NULL,

CONSTRAINT pk_tb_triagem_id_triagem
PRIMARY KEY(id_triagem),

CONSTRAINT fk_tb_triagem_id_acolhido
FOREIGN KEY(id_acolhido)
REFERENCES tb_acolhido(id_acolhido),

CONSTRAINT fk_tb_triagem_id_inst_parceira
FOREIGN KEY(id_inst_parceira)
REFERENCES tb_instituicao_parceira(id_inst_parceira),

CONSTRAINT fk_tb_triagem_id_usuario_resp
FOREIGN KEY(id_usuario_resp)
REFERENCES tb_usuarios(id_usuario)
);

CREATE TABLE tb_acompanhamento_diario (
id_acomp             INTEGER,
id_acolhido           INTEGER,
id_usuario_resp       INTEGER,
data_registro         DATE          NOT NULL,
comportamento        TEXT,
atividades_realizadas TEXT,
progresso_emprego     TEXT,
observacoes           VARCHAR(500),

CONSTRAINT pk_tb_acompanhamento_diario
PRIMARY KEY(id_acomp),

CONSTRAINT fk_tb_acomp_id_acolhido
FOREIGN KEY(id_acolhido)
REFERENCES tb_acolhido(id_acolhido),

CONSTRAINT fk_tb_acomp_id_usuario_resp
FOREIGN KEY(id_usuario_resp)
REFERENCES tb_usuarios(id_usuario)
);

```

```

CREATE TABLE tb_empresa_parceira (
id_empresa      INTEGER,
nome            VARCHAR(150) NOT NULL,
cnpj            CHAR(14)      UNIQUE NOT NULL,
segmento        VARCHAR(100),
telefone        VARCHAR(20),
email           VARCHAR(150),
endereco        VARCHAR(255),
ativo           BOOLEAN       DEFAULT TRUE,

CONSTRAINT pk_tb_empresa_parceira
PRIMARY KEY(id_empresa)
);

```

```

CREATE TABLE tb_oportunidade_emprego (
id_oportunidade INTEGER,
id_empresa       INTEGER,
cargo            VARCHAR(120) NOT NULL,
descricao        TEXT,
requisitos       TEXT,
tipo_contrato    VARCHAR(80),
carga_horaria    VARCHAR(60),
data_publicacao  DATE          NOT NULL,
data_encerramento DATE,
vagas_disponiveis INTEGER       NOT NULL,
ativo           BOOLEAN       DEFAULT TRUE,

```

```

CONSTRAINT pk_tb_oportunidade_emprego
PRIMARY KEY(id_oportunidade),

```

```

CONSTRAINT fk_tb_oportunidade_id_empresa
FOREIGN KEY(id_empresa)
REFERENCES tb_empresa_parceira(id_empresa)
);

```

```

CREATE TABLE tb_acolhido_oportunidade (
id_acolhido      INTEGER,
id_oportunidade  INTEGER,

```

```

CONSTRAINT pk_tb_acolhido_oportunidade
PRIMARY KEY(id_acolhido, id_oportunidade),

```

```

CONSTRAINT fk_tb_acolhido_oportunidade_acolhido
FOREIGN KEY(id_acolhido)
REFERENCES tb_acolhido(id_acolhido),

```

```

CONSTRAINT fk_tb_acolhido_oportunidade_oportunidade
FOREIGN KEY(id_oportunidade)
REFERENCES tb_oportunidade_emprego(id_oportunidade)
);

```

```

CREATE TABLE tb_relatorio_mensal (
id_relatorio      INTEGER,
id_acolhido        INTEGER,
mes_referencia     INTEGER       NOT NULL,
ano_referencia     INTEGER       NOT NULL,
resumo_atividades  TEXT,
evolucao           TEXT,
encaminhamentos    TEXT,
gerado_em          DATE          NOT NULL,
id_usuario_gerador INTEGER,

```



```

CONSTRAINT pk_tb_relatorio_mensal
PRIMARY KEY(id_relatorio),

CONSTRAINT fk_tb_relatorio_id_acolhido
FOREIGN KEY(id_acolhido)
REFERENCES tb_acolhido(id_acolhido),

CONSTRAINT fk_tb_relatorio_id_usuario_gerador
FOREIGN KEY(id_usuario_gerador)
REFERENCES tb_usuarios(id_usuario)
);

CREATE TABLE tb_doador (
id_doador    INTEGER,
nome         VARCHAR(150) NOT NULL,
cpf_cnpj     VARCHAR(18)  UNIQUE,
tipo         VARCHAR(20)  NOT NULL
            CHECK(tipo IN ('pessoa_fisica', 'empresa')),
telefone     VARCHAR(20),
email        VARCHAR(150),
endereco     VARCHAR(255),
ativo        BOOLEAN      DEFAULT TRUE,

CONSTRAINT pk_tb_doador
PRIMARY KEY(id_doador)
);

CREATE TABLE tb_doacao (
id_doacao    INTEGER,
id_doador    INTEGER,
data_doacao  DATE        NOT NULL,
tipo         VARCHAR(20)  NOT NULL
            CHECK(tipo IN ('financeira', 'material')),
valor        DECIMAL(12,2),
descricao    VARCHAR(500),
comprovante_path VARCHAR(255),

CONSTRAINT pk_tb_doacao
PRIMARY KEY(id_doacao),

CONSTRAINT fk_tb_doacao_id_doador
FOREIGN KEY(id_doador)
REFERENCES tb_doador(id_doador)
);

CREATE TABLE tb_financas (
id_financa   INTEGER,
id_doacao    INTEGER,
data_lancamento DATE    NOT NULL,
tipo         VARCHAR(20)  NOT NULL
            CHECK(tipo IN ('receita', 'despesa')),
categoria    VARCHAR(100),
descricao    VARCHAR(500),
valor        DECIMAL(12,2) NOT NULL,
forma_pagamento VARCHAR(60),
observacoes  VARCHAR(500),

CONSTRAINT pk_tb_financas
PRIMARY KEY(id_financa),

CONSTRAINT fk_tb_financas_id_doacao
FOREIGN KEY(id_doacao)
REFERENCES tb_doacao(id_doacao)
);

```

```

CREATE TABLE tb_item_estoque (
id_item          INTEGER,
nome             VARCHAR(150) NOT NULL,
categoria        VARCHAR(100),
unidade         VARCHAR(20)  NOT NULL,
quantidade_atual INTEGER     NOT NULL,
quantidade_minima INTEGER     NOT NULL,
local_armazenamento VARCHAR(100),
ativo           BOOLEAN      DEFAULT TRUE,

CONSTRAINT pk_tb_item_estoque
PRIMARY KEY(id_item)
);

```

```

CREATE TABLE tb_alerta_estoque (
id_alerta        INTEGER,
id_item          INTEGER,
data_alerta      DATE      NOT NULL,
quantidade_atual INTEGER NOT NULL,
quantidade_minima INTEGER NOT NULL,
observacoes      VARCHAR(500),
resolvido        BOOLEAN DEFAULT FALSE,
data_resolucao   DATE,

CONSTRAINT pk_tb_alerta_estoque
PRIMARY KEY(id_alerta),

CONSTRAINT fk_tb_alerta_estoque_id_item
FOREIGN KEY(id_item)
REFERENCES tb_item_estoque(id_item)
);

```

```

CREATE TABLE tb_controle_vagas (
id_controle_vaga INTEGER,
data_referencia  DATE      NOT NULL,
vagas_totais     INTEGER NOT NULL,
vagas_ocupadas   INTEGER NOT NULL,
vagas_disponiveis INTEGER NOT NULL,
observacoes      VARCHAR(500),

CONSTRAINT pk_tb_controle_vagas
PRIMARY KEY(id_controle_vaga)
);

```

4. Matriz de rastreabilidade entre requisitos e implementação

ID	Requisito	Funcionalidade	Tela / Módulo	Classe / Arquivo / Tabela
RF01	Realizar login	Autenticação de usuário	LoginView	AuthController.java / tb_usuarios
RF02	Cadastrar / gerenciar usuários	Gestão de usuários	DashboardView	Usuario.java / UsuarioDAO.java / tb_usuarios
RF03	Gerenciar perfis de acesso	Controle de permissões	DashboardView	Perfil.java / PerfilDAO.java / tb_perfil

ID	Requisito	Funcionalidade	Tela / Módulo	Classe / Arquivo / Tabela
RF04	Cadastrar acolhido	Admissão de acolhido	AcolhidoView	Acolhido.java / AcolhidoDAO.java / tb_acolhido
RF05	Realizar triagem	Triagem de acolhido	TriagemView	Triagem.java / TriagemDAO.java / tb_triagem
RF06	Registrar acompanhamento diário	Acompanhamento de acolhido	AcompanhamentoView	AcompanhamentoDiario.java / tb_acompanhamento_diario
RF07	Cadastrar instituição parceira	Gestão de parcerias	InstituicaoView	InstituicaoParceira.java / tb_instituicao_parceira
RF08	Cadastrar empresa parceira	Gestão de empresas	EmpresaView	EmpresaParceira.java / tb_empresa_parceira
RF09	Gerenciar oportunidades de emprego	Recolocação profissional	OportunidadeView	OportunidadeEmprego.java / tb_oportunidade_emprego
RF10	Gerar relatório mensal	Relatórios de acompanhamento	RelatorioView	RelatorioMensal.java / tb_relatorio_mensal
RF11	Cadastrar doador	Gestão de doadores	DoacaoView	Doador.java / DoadorDAO.java / tb_doador
RF12	Registrar doação	Controle de doações	DoacaoView	Doacao.java / DoacaoDAO.java / tb_doacao
RF13	Controlar finanças	Gestão financeira	FinancaView	Financa.java / FinancaDAO.java / tb_financas
RF14	Gerenciar estoque	Controle de estoque	EstoqueView	ItemEstoque.java / tb_item_estoque
RF15	Alertar estoque mínimo	Alerta automático de estoque	EstoqueView	AlertaEstoque.java / tb_alerta_estoque
RF16	Controlar vagas	Gestão de vagas disponíveis	VagasView	ControleVagas.java / tb_controle_vagas

5. README técnico

Arquivo do Readme está no Git e também em anexo.